
Discovering Cooperative Pipelines: Autoresearch for Sequential Social Dilemmas

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We study *two-level autoresearch* for cooperation: an outer-loop AI agent au-
2 tonomously redesigns the inner-loop pipeline of an LLM policy-synthesis sys-
3 tem for multi-agent Sequential Social Dilemmas (SSDs). A *researcher agent* $\mathcal{R}$
4 (run as a coding agent) reads the inner-loop source code, edits system prompts,
5 feedback functions, helper libraries, and iteration logic, runs evaluations, and
6 decides what to keep, following the *autoresearch* paradigm. Across two games
7 (Cleanup and Gathering), two policy-synthesizer LLMs, and two welfare objec-
8 tives (utilitarian efficiency and Rawlsian maximin), the researcher reliably exceeds
9 hand-designed baselines, sharply tightens run-to-run variance, and outperforms
10 prompt-only optimization. The discovered pipelines are objective-dependent: un-
11 der maximin the researcher independently rediscovers *duty rotation* as a fairness
12 mechanism, supporting an *information-design* reading in which the researcher
13 chooses what to reveal to the boundedly rational synthesizer. Code at <https://github.com/anon590/llm-policies-social-dilemmas>.
14

15 1 Introduction

16 Sequential Social Dilemmas (SSDs) [1] are the multi-agent analogue of the prisoner’s dilemma in
17 temporally rich Markov games: individually rational play leads to collectively suboptimal outcomes
18 through pollution, over-harvesting, or open conflict. Standard multi-agent reinforcement learning
19 (MARL) struggles in this regime due to credit assignment, non-stationarity, and large joint action
20 spaces [2]. A complementary approach, recently introduced by Gallego [3], replaces parameter-
21 space optimization with *algorithm-space synthesis*: a frozen LLM writes a Python policy function,
22 evaluates it in self-play, and iteratively refines it from performance feedback. A single generation
23 step can produce coordination logic (territory partitioning, role assignment, conditional cooperation)
24 at a sample efficiency several orders of magnitude beyond what gradient-based MARL achieves on
25 the same environments.

26 This shifts where the design problem lives, rather than removing it. The inner-loop pipeline that
27 drives the synthesizer has many free parameters: which system prompt, which feedback variables,
28 which helper functions, how many refinement steps. Each materially affects the resulting policies,
29 and prior work tuned them by hand. A natural question follows: *can an AI agent design the pipeline?*

30 We answer affirmatively with a two-level autoresearch framework. An *outer-loop researcher agent*
31 (Claude Opus 4.6, run as a coding agent) edits the source files of an *inner-loop policy synthesizer*
32 (another LLM), runs evaluations on held-out seeds, and keeps modifications that improve a fixed
33 welfare objective Φ . The outer agent operates on an ordinary git repository (reading code, writing
34 diffs, running shell commands, etc) without task-specific scaffolding, mirroring the autoresearch
35 paradigm of Karpathy [4] for single-GPU LLM pretraining. Although the inner-loop SSDs are
36 gridworld benchmarks rather than physical systems, the outer-loop discovery process itself runs

under conditions a deployed discovery agent faces: noisy multi-seed evaluations, stochastic code generation, an LLM-evaluation budget that bounds how often Φ can be queried, and a heterogeneous code repository the agent must navigate end-to-end on its own.

Our contributions are: i) a general two-level framework that delegates the design of an LLM synthesis pipeline to a coding agent operating on a real software repository (Section 3); ii) the first instantiation of the autoresearch paradigm in a multi-agent decision-making domain, with experiments across two SSDs (Cleanup and Gathering), two policy LLMs, and two welfare objectives (utilitarian efficiency U and Rawlsian maximin $\min_i R_i$) (Section 4); and iii) a mechanism-design interpretation supported by the qualitatively different pipelines the agent produces under different welfare objectives, including the autonomous discovery of *time-based duty rotation* as a fairness mechanism, never found under efficiency optimization.

2 Background

We build on the iterative LLM policy synthesis framework of Gallego [3], which serves as the (frozen) inner loop of our two-level system. This section recalls the SSD formalism, the social outcome metrics, and the base synthesis loop.

2.1 Sequential Social Dilemmas

A Sequential Social Dilemma is a partially observable Markov game $\mathcal{G} = \langle N, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^N, T, \{R_i\}_{i=1}^N, H \rangle$ with N agents, state space $\mathcal{S}$ (the gridworld configuration), per-agent action spaces $\mathcal{A}_i$, transition function T , reward functions R_i , and episode horizon H [1]. Beyond the dilemma’s matrix-game structure, SSDs add temporal richness: agents must learn *when* and *where* to cooperate, not just *whether* to.

We study two canonical SSDs that capture complementary dilemma types.

Cleanup [5] is a *public goods provision* game ($N=10$). The map has two regions: a river that accumulates waste, and an orchard where apples grow. Apples regrow only when river pollution is below threshold. Each agent can fire a cleaning beam (cost -1) that removes waste, collect apples ($+1$ each), or fire a tagging beam (cost -1 , inflicting -50 on the target and removing it for 25 steps). The dilemma: cleaning is costly but its benefits are public, so purely self-interested agents free-ride.

Gathering [1, 6] is a *common pool resource* game ($N=4$). Agents collect shared apples on a fixed respawn timer and may fire tagging beams to temporarily remove rivals. The dilemma: agents can coexist and share resources, or aggress to monopolize them; aggression wastes time and reduces total welfare.

Both games use 8–9 discrete actions (movement, rotation, beam, stand, optionally clean) and episodes of $H=1000$ steps. The two dilemmas differ in cost structure: asymmetric provision (cleaners pay, all benefit) vs. symmetric restraint (every agent faces the same temptation), a distinction that drives our experimental findings.

Social metrics. Following Perolat et al. [6], let $R_i = \sum_{t=0}^{H-1} r_i^t$ denote agent i ’s episode return. We evaluate four social outcomes:

$$U = \frac{1}{H} \sum_{i=1}^N R_i \quad (\text{efficiency}) \quad (1)$$

$$E = 1 - \frac{\sum_{i,j} |R_i - R_j|}{2N \sum_i R_i} \quad (\text{equality}) \quad (2)$$

$$S = \frac{1}{N} \sum_{i=1}^N \bar{t}_i \quad (\text{sustainability}) \quad (3)$$

$$P = \frac{1}{H} \sum_{t=0}^{H-1} |\{i : \text{active}_i^t\}| \quad (\text{peace}) \quad (4)$$

where $\bar{t}_i$ is the mean timestep at which agent i collects positive reward (higher $\Rightarrow$ resources preserved later) and active_i^t indicates that agent i is not tagged out at step t . We additionally consider the

76 *maximin* (Rawlsian) welfare criterion $\min_i R_i$, which optimizes the worst-off agent’s return and
 77 serves as the second objective in our experiments.

78 2.2 Iterative LLM Policy Synthesis

79 Let Π denote the space of *code-based policies*: deterministic functions $\pi : \mathcal{S} \times [N] \rightarrow \mathcal{A}$ expressed
 80 as executable Python code. Each policy has access to the full environment state and a library of
 81 helpers (BFS pathfinding, beam targeting, coordinate transforms). This state access is a deliberate
 82 design choice: programmatic policies operate in algorithm space rather than the reactive observation-
 83 to-action space of neural policies, which lets a single LLM generation step encode rich coordination
 84 logic.

85 A frozen LLM $\mathcal{M}$ acts as the *policy synthesizer*. Given a system prompt p describing the environ-
 86 ment API and a feedback prompt q_k , it produces a new policy

$$\pi_{k+1} = \mathcal{M}(p, q(\pi_k, \mathcal{F}_k)),$$

87 where π_k is the previous policy (its source code) and $\mathcal{F}_k$ is the evaluation feedback. All N agents ex-
 88 ecute the same policy in homogeneous self-play, and evaluation over a set of random seeds S yields
 89 the mean per-agent return $\bar{r}_k$ and the social metrics vector $\mathbf{m}_k = (U_k, E_k, S_k, P_k)$. Each generated
 90 policy passes an AST-based safety check (blocking eval, file I/O, network access) followed by a
 91 short smoke test; failures trigger regeneration (up to R attempts) with the error message appended
 92 to the prompt.

93 **Feedback.** The hand-designed feedback used by Gallego [3] as our starting point packages the
 94 previous policy’s source code together with all available evaluation signals:

$$\mathcal{F}_k = (\text{code}(\pi_k), \bar{r}_k, \mathbf{m}_k, \mathbf{d}), \quad (5)$$

95 where $\mathbf{d}$ contains natural-language definitions of each social metric. Exposing the full social-metrics
 96 vector (rather than the scalar reward alone) disambiguates distinct failure modes that share the same
 97 average return (e.g., under- vs. over-cleaning in Cleanup) and consistently improves cooperative
 98 outcomes. On Cleanup, this baseline feedback reaches $U=1.93/2.70$ (mean/max) with Gemini and
 99 $U=0.86/1.56$ with Sonnet across 4 runs each. The choice of feedback content is a single design
 100 decision within a much larger pipeline configuration space; our two-level framework (Section 3)
 101 opens the full space to automated search.

102 3 Two-Level Framework

103 We introduce a two-level system where a *researcher agent* $\mathcal{R}$ autonomously discovers configurations
 104 that optimize the output of an inner-loop system. While we instantiate this for multi-agent policy
 105 synthesis, the architecture is general: any pipeline where an LLM generates artifacts, evaluates them,
 106 and iterates can serve as the inner loop. The key insight is that the entire inner-loop codebase is a
 107 designable artifact that an AI agent can search over. Figure 1 illustrates the architecture.

108 3.1 Configuration Space

109 Let $\mathcal{C}$ denote the space of *pipeline configurations*. Each configuration $c \in \mathcal{C}$ specifies the full inner-
 110 loop setup:

$$c = (p, \phi, \mathcal{H}, \iota, v) \quad (6)$$

111 where p is the system prompt, ϕ is the feedback construction function (which metrics and diagnostics
 112 to include, how to frame it, whether to inject adaptive hints), $\mathcal{H}$ is the helper function library, ι
 113 specifies the iteration logic (number of iterations K , sampling strategy, temperature schedule), and
 114 v is the validation pipeline. Table 1 provides concrete examples.

115 The hand-designed feedback of [3] corresponds to a single fixed instantiation of ϕ . Our framework
 116 opens the full configuration space to automated search.

117 3.2 Inner Loop (Policy Synthesis)

118 Given a configuration c , the inner loop executes K iterations of LLM policy synthesis:

$$\pi_c^* = \text{INNERLOOP}(\mathcal{M}, \mathcal{G}, c) \quad (7)$$

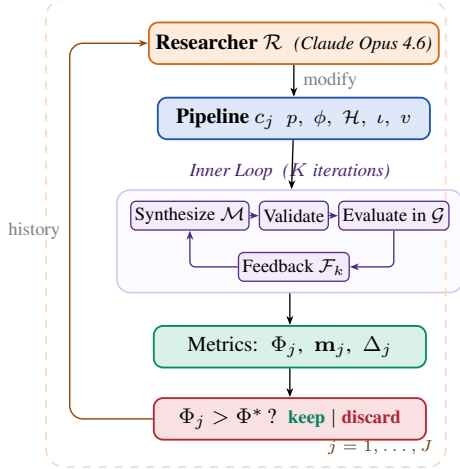


Figure 1: Two-level automated research framework (Algorithm 1).

Algorithm 1 Two-Level Automated Research

Require: Game $\mathcal{G}$, policy synthesizer $\mathcal{M}$, researcher $\mathcal{R}$, system prompt $p_{\mathcal{R}}$, initial config c_0 , outer iterations J , ground-truth objective Φ , held-out seeds S_{ho}

Ensure: Best configuration c^* , best policy π^*

```

1:  $\pi_0^* \leftarrow \text{INNERLOOP}(\mathcal{M}, \mathcal{G}, c_0)$ 
2:  $\Phi_0 \leftarrow \text{EVAL}(\pi_0^*; \mathcal{G}, S_{\text{ho}})$ 
3:  $\text{history} \leftarrow \{(c_0, \Phi_0, \mathbf{m}_0, \emptyset)\}$ 
4: for  $j = 1, \dots, J$  do
5:    $c_j \leftarrow \mathcal{R}(p_{\mathcal{R}}, \text{CODE}(c_{j-1}), \text{history})$ 
6:   if  $\neg \text{VALIDATECONFIG}(c_j)$  then  $\text{retry} \leq R$ 
7:      $\pi_j^* \leftarrow \text{INNERLOOP}(\mathcal{M}, \mathcal{G}, c_j)$ 
8:      $\Phi_j \leftarrow \text{EVAL}(\pi_j^*; \mathcal{G}, S_{\text{ho}})$ 
9:      $\Delta_j \leftarrow \text{DIFF}(c_{j-1}, c_j)$  // code diff
10:     $\text{history} \leftarrow \text{history} \cup \{(c_j, \Phi_j, \mathbf{m}_j, \Delta_j)\}$ 
11:  end for
12:  $j^* \leftarrow \arg \max_j \Phi_j$ 
13: return  $c_{j^*}, \pi_{j^*}^*$ 

```

Table 1: Configuration components modifiable by the researcher, with concrete edits that $\mathcal{R}$ made in our runs. The environment simulator, ground-truth evaluation, and policy LLM weights are frozen.

Scope	Concrete edits by $\mathcal{R}$ (see appendix)	
p	System prompt	Multi-section strategic briefing with explicit duty-rotation template (<code>agent_id + step//T</code>) % n (App. D.2, Listing 3)
ϕ	Feedback fn	Thresholded diagnostics: “FAIRNESS ALERT” fires when $\min_i R_i < 0$; “DO NOT REGRESS” guard fires when $U \geq 2.5$ (App. D.3, Listing 4)
$\mathcal{H}$	Helper library	BFS-Voronoi territories with respawn-timer awareness; band-based apple zoning by <code>agent_id</code> (App. D.1, Listings 2, 1)
ι	Iteration logic	Per-condition $K \in \{2, 3\}$; $ S $ walked $5 \rightarrow 8 \rightarrow 12$ for variance control; thinking budget 10–32k (App. D.4, Table 6)
v	Validation	AST blocks <code>eval/file</code> I/O/network; 50-step smoke test; $\text{retry} \leq R$ on failure

Each iteration k proceeds in four stages, following [3]:

1. **Synthesize.** The policy LLM $\mathcal{M}$ receives the system prompt p , the previous policy’s source code π_{k-1} , and feedback $\mathcal{F}_{k-1}$ constructed by ϕ . It generates a new Python policy function π_k that has access to full environment state and the helper library $\mathcal{H}$.
2. **Validate.** The generated code undergoes AST-based safety checks (blocking dangerous operations such as file I/O and network access) followed by a short smoke test. Failures trigger re-generation (up to R retries), with the error message appended to the prompt.
3. **Evaluate.** All N agents execute the same policy π_k in self-play over $|S|$ random seeds. The evaluation yields the mean per-agent reward $\bar{r}_k$ and the social metrics vector $\mathbf{m}_k = (U_k, E_k, S_k, P_k)$.
4. **Feedback.** The feedback function ϕ constructs the prompt for the next iteration from $(\pi_k, \bar{r}_k, \mathbf{m}_k)$, packaging the previous policy’s source code together with the scalar reward, the social metrics vector, their natural-language definitions, and any adaptive diagnostics that ϕ injects.

The inner loop output is evaluated on held-out seeds via a fixed ground-truth objective:

$$\Phi(c) = \text{EVAL}(\pi_c^*; \mathcal{G}, S_{\text{held-out}}) \quad (8)$$

where Φ maps from social outcomes to a scalar. We consider two objectives:

$$\Phi_U = U \quad (\text{utilitarian efficiency}) \quad (9)$$

$$\Phi_{\min} = \min_i R_i \quad (\text{Rawlsian maximin}) \quad (10)$$

3.3 Outer Loop (Automated Research)

The researcher agent $\mathcal{R}$ iteratively modifies the pipeline configuration. Following the autoresearch paradigm [4], $\mathcal{R}$ operates on the inner-loop codebase as a modifiable artifact, proposing changes, observing outcomes, and refining. The procedure is formalized in Algorithm 1.

At each outer iteration j , the researcher $\mathcal{R}$ receives: i) the **full source code** of the current configuration c_{j-1} (prompts, feedback construction, helpers, iteration logic); ii) the **experiment history**: for each prior iteration, the code diff Δ_i , ground-truth score Φ_i , and social metrics vector $\mathbf{m}_i$; iii) the **environment source code** (read-only), enabling the researcher to reason about game mechanics. The researcher proposes a new configuration c_j by generating code modifications. Concretely, $\mathcal{R}$ is a coding agent (Claude Code CLI) that operates on a real software repository: it reads and edits Python source files, runs shell commands, inspects evaluation outputs, and commits changes to a dedicated git branch, following the same workflow a human researcher would follow. This mirrors the original autoresearch setup [4], where an AI coding agent modifies `train.py` and observes validation loss; here, the researcher modifies the inner-loop codebase and observes social welfare.

3.4 Connection to Automated Mechanism Design

The two-level structure admits a mechanism design interpretation. The researcher $\mathcal{R}$ acts as a *mechanism designer*: it controls the information structure (what metrics to reveal, how to frame them), the action space (what helper functions are available), and the incentive structure (how feedback is presented) under which the policy synthesizer $\mathcal{M}$ operates. The synthesizer acts as the *agent* within the designed mechanism.

This connects to the automated mechanism design literature [7], where a principal designs rules to induce desired behavior from self-interested agents. In our setting:

- The **principal** is the researcher $\mathcal{R}$, optimizing the ground-truth objective Φ .
- The **agent** is the synthesizer $\mathcal{M}$, optimizing per-agent reward as instructed.
- The **mechanism** is the configuration c : prompts, feedback, helpers, iteration logic.
- The **outcome** is the social welfare of the resulting multi-agent policy π_c^* .

A crucial distinction from classical mechanism design: $\mathcal{M}$ follows instructions but has *bounded rationality* in the sense that its ability to synthesize effective policies depends on the information and tools provided. The researcher’s task is thus closer to *information design* [8]: choosing what to reveal to help $\mathcal{M}$ navigate the cooperation–defection tension. Our experiments (Section 4) show that the researcher designs qualitatively different information structures depending on the welfare objective Φ , supporting this interpretation empirically.

4 Experiments

4.1 Setup

We conduct 12 autonomous researcher runs across a factorial design. The researcher agent $\mathcal{R}$ is Claude Opus 4.6, invoked via the Claude Code CLI as a coding agent. Each run operates on a dedicated git branch of a real Python codebase: $\mathcal{R}$ edits source files in `pipeline/`, executes evaluation scripts, reads metric outputs, and iterates, without human intervention.

Design. For Cleanup: 2 policy LLMs $\times$ 2 objectives $\times$ 2 replications = 8 runs. For Gathering: 2 policy LLMs $\times$ 1 objective $\times$ 2 replications = 4 runs. Maximin runs are unnecessary for Gathering because efficiency optimization alone achieves equality $E > 0.94$ (the game’s symmetric cost structure makes fairness a free byproduct of coordination).

Models. Policy synthesizer $\mathcal{M}$: Gemini 3.1 Pro (Google) or Claude Sonnet 4.6 (Anthropic). Both use extended thinking. These are the same models evaluated in [3]. We additionally test with Gemma 4 26B-A4B-IT (Google), a smaller open-weight model, to probe the framework’s behavior when the policy synthesizer has substantially lower capability (Appendix C).

Baselines. The hand-designed feedback configuration from [3] serves as the initial pipeline c_0 for all runs. On Cleanup ($N=10$), this baseline achieves $U=1.93/2.70$ (mean/max) with Gemini

Table 2: Results. Mean / max across 2 runs per condition (4 for Cleanup baselines). Baseline: unmodified pipeline from [3]. GEPA: automated prompt optimization [9] with matched compute budget.

Policy LLM	Target	U	E	$\min_i R_i$
<i>Cleanup (N=10) — Baseline</i>				
Gemini 3.1 Pro	—	1.93/2.70	0.17/0.62	−159/−84
Sonnet 4.6	—	0.86/1.56	−0.02/0.25	−151/−59
<i>Cleanup (N=10) — GEPA [9]</i>				
Gemini 3.1 Pro	Φ_U	1.34/1.37	−0.10/−0.05	−149/−126
	$\Phi_{\min}$	1.76/2.76	0.90/0.96	143/245
Sonnet 4.6	Φ_U	1.04/1.20	−0.59/−0.21	−164/−126
	$\Phi_{\min}$	−0.63/1.60	0.77/1.00	−147/6
<i>Cleanup (N=10) — Autoresearch</i>				
Gemini 3.1 Pro	Φ_U	3.20 /3.25	0.55/0.61	−211/−182
	$\Phi_{\min}$	3.16/3.19	0.98 /0.98	290 /296
Sonnet 4.6	Φ_U	3.12 /3.14	0.66/0.70	−196/−146
	$\Phi_{\min}$	2.57/2.93	0.91 /0.97	179 /200
<i>Gathering (N=4) — Baseline</i>				
Gemini 3.1 Pro	—	2.04/2.42	0.90/0.98	412/571
Sonnet 4.6	—	0.03/0.03	0.54/0.54	0/0
<i>Gathering (N=4) — GEPA [9]</i>				
Gemini 3.1 Pro	Φ_U	2.08/2.35	0.94/0.96	436/518
Sonnet 4.6	Φ_U	1.20/1.23	0.63/0.71	86/123
<i>Gathering (N=4) — Autoresearch</i>				
Gemini 3.1 Pro	Φ_U	2.49/2.51	0.98/0.98	582/582
Sonnet 4.6	Φ_U	2.52/2.52	0.96/0.98	576/597

and $U=0.86/1.56$ with Sonnet across 4 runs each. We additionally compare against GEPA [9], an automated prompt optimization method that iteratively refines the system prompt via LLM reflection. GEPA optimizes only the system prompt p , whereas our researcher modifies the full pipeline $(p, \phi, \mathcal{H}, \iota, v)$. We give GEPA a matched compute budget: the same number of optimization steps as outer iterations used by our method (each GEPA step costs ~ 3 policy evaluations, comparable to our inner loop’s $K+1=4$).

Inner loop parameters. $K=3$ synthesis iterations (or $K=2$ if the researcher elects), evaluated over $|S|=5$ –12 random seeds (researcher-configurable). The researcher ran $J=3$ –17 outer iterations per run (mean: 8.3), with a keep rate of 18%–100% (mean: 63%).

4.2 Results

Table 2 presents the main results, comparing autoresearch against the hand-designed baseline of [3] and GEPA prompt optimization [9].

Finding 1: The researcher reliably improves over hand-designed baselines and outperforms prompt-only optimization. All 12 runs show substantial improvement regardless of starting point (Figure 2). In Cleanup, the researcher exceeds the unmodified pipeline baseline by 66% with Gemini ($U: 1.93 \rightarrow 3.20$) and 263% with Sonnet ($U: 0.86 \rightarrow 3.12$). Strikingly, the Sonnet improvement nearly closes the gap with Gemini ($U=3.12$ vs. 3.20), despite a $2\times$ baseline disadvantage (0.86 vs. 1.93). Final efficiency values cluster tightly around $U \approx 3.1$ –3.2 despite baselines varying from 0.29 to 2.70 across individual runs, suggesting the researcher reliably discovers the performance ceiling of each policy LLM. Beyond improving the mean, the researcher yields remarkably consistent results: under efficiency optimization, Gemini’s runs cluster within $U=3.20$ –3.25 (gap 0.05) despite baselines spanning 1.15–2.70, and Sonnet’s within 3.12–3.14 (gap 0.02) from baselines of 0.38–1.56. In Gathering, all four runs converge to $U=2.47$ –2.52 from baselines ranging 0.03–2.42. The researcher discovers pipeline modifications: structured helpers (App. D.1), strategic hints (App. D.2), tailored feedback (App. D.3). These constrain the policy LLM toward consistently good solutions.

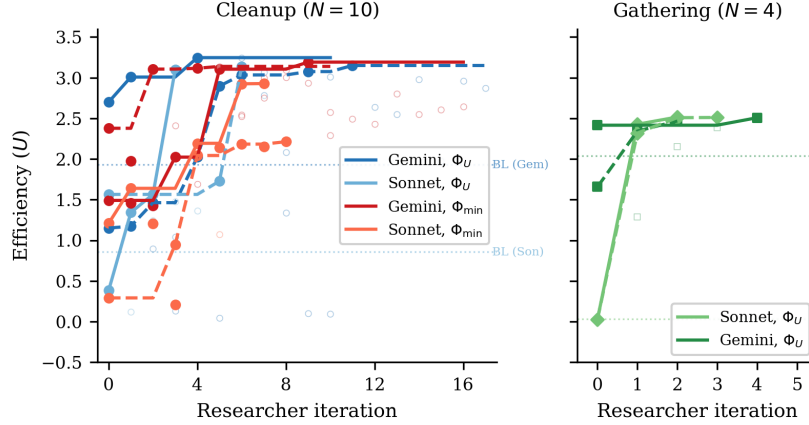


Figure 2: Efficiency (U) across researcher iterations for all 12 runs. *Left*: Cleanup ($N=10$) with 8 runs across 2 LLMs $\times$ 2 objectives (solid lines = kept experiments, open circles = discarded). Dashed horizontal line: hand-designed baseline from [3]. All runs converge to $U \approx 3.1$ – 3.2 despite diverse starting points. *Right*: Gathering ($N=4$) with 4 runs (2 per LLM). All converge to $U \approx 2.5$ within 2–4 iterations despite baselines ranging from 0.03 to 2.42.

208 Compared to GEPA (Table 2), autoresearch achieves $2.4\times$ higher efficiency under Φ_U in Cleanup
 209 ($U=3.20$ vs. 1.34), $2\times$ higher maximin under $\Phi_{\min}$ (290 vs. 143), and 20% higher efficiency in Gath-
 210 ering ($U=2.49$ vs. 2.08). Critically, GEPA shows far wider spread across runs on the harder fairness
 211 objective ($\Phi_{\min}$ maximin in Cleanup: $245/41$ vs. autoresearch’s $296/284$). The gap widens further
 212 with Sonnet: on Cleanup Φ_U , GEPA stalls at $U=1.04$ ($3\times$ below autoresearch’s 3.12), and on $\Phi_{\min}$
 213 its two runs split between a modestly positive solution ($\min_i R_i=6$, $U=1.60$) and a pathological “ev-
 214 eryone cleans, nobody eats” regime (perfect equality $E=1.00$ but negative efficiency $U=-2.87$ and
 215 $\min_i R_i=-299$), while autoresearch with the same model reaches $\min_i R_i=179/200$ reliably. This
 216 highlights the advantage of modifying the full pipeline, whereas prompt-only optimization cannot
 217 address the underlying stochasticity of code generation.

218 **Finding 2: No efficiency–fairness tradeoff in Cleanup (Gemini).** Maximin-optimized Gem-
 219 ini pipelines sacrifice only 1% efficiency (U : 3.16 vs. 3.20) while achieving near-perfect equal-
 220 ity (E : 0.98 vs. 0.55) and transforming maximin from deeply negative baselines (-99 to -84) to
 221 $\min_i R_i = 290$. The researcher discovers *fair duty rotation* (Listing 6; primed by the prompt rewrite
 222 of Listing 3)—time-based cycling using `agent_id` and `env._step_count`—simultaneously im-
 223 proving worst-off welfare *and* collective output. Because cleaning is a public good, distributing the
 224 cleaning cost fairly ensures enough cleaners to sustain apple production.

225 For Sonnet, there is a moderate tradeoff: efficiency drops from 3.12 to 2.57 under maximin opti-
 226 mization. The gap reflects Sonnet’s harder time implementing complex coordination mechanisms
 227 (role rotation, zone assignment) from strategic hints alone.

228 **Finding 3: Game structure determines whether fairness requires explicit optimization.** In
 229 Cleanup, where cleaning costs are borne asymmetrically (cleaners pay -1 , free-riders collect ap-
 230 ples), baseline equality ranges from $E=0.04$ to 0.62 , and maximin optimization is required to reach
 231 $E > 0.9$. In Gathering, where all agents face a symmetric landscape, efficiency optimization alone
 232 achieves $E > 0.94$ across all 4 runs: no separate maximin runs are needed.

233 This generalizes: in *provision* dilemmas with asymmetric costs, fairness requires designed mecha-
 234 nisms (role rotation, duty sharing); in *restraint* dilemmas with symmetric costs, fairness emerges as a
 235 free byproduct of efficient coordination. The researcher independently discovers this, it creates role
 236 differentiation pipelines only for Cleanup, and pure spatial-coordination pipelines for Gathering.

237 **Finding 4: Convergent discovery of qualitatively different strategies per objective.** Despite
 238 fully independent runs, the researcher converges on the same core strategies within each condition
 239 (Table 3, Appendix A). In Cleanup, waste-counting helpers and spatial zone partitioning appear

across all 8 runs. The key qualitative difference is *role rotation*, appearing in 4/4 maximin runs (Listings 6, 7) but 0/4 efficiency runs (Listing 5). Under efficiency optimization, the researcher assigns static cleaning roles (some agents always clean), achieving high collective output at the cost of equality. Under maximin optimization, it discovers rotation as a fairness mechanism.

In Gathering, the researcher discovers BFS-Voronoi territory partitioning and respawn-timer awareness (Listings 2, 8), with no role differentiation. Thus, optimization is purely spatial (who collects which apples) and temporal (respawn-aware positioning).

Common failure modes. Several patterns led to discarded experiments across all runs: (1) *over-prescription*: adding too many strategic hints confuses the policy LLM, causing generation failures; (2) *iteration regression*: $K=4$ or $K=5$ inner iterations often cause catastrophic regression as the LLM “over-refines” a working policy; (3) *feedback overload*: verbose per-agent diagnostics cause over-correction rather than gradual improvement; (4) *variance exposure*: identical configurations can yield $U=3.2$ then $U=0.09$ on different seeds, requiring multi-seed evaluation for stable signals (per-run $K, |S|$ in Table 6).

5 Related Work

Sequential social dilemmas. SSDs were introduced by Leibo et al. [1] (Gathering) and extended to public-goods settings by Hughes et al. [5] (Cleanup); Perolat et al. [6] formalized the social outcome metrics (U, E, S, P) used here. Subsequent work studies inequity aversion, intrinsic motivation, and reputational mechanisms for promoting cooperation in MARL agents. We complement this line by automating the search for cooperative *programs* rather than evolving neural policies, and by treating the welfare objective Φ as a designable parameter that the outer agent optimizes for, obtaining qualitatively different cooperative behaviors (static role assignment vs. duty rotation) under different objectives without changing the environment.

LLMs for policy and program synthesis. FunSearch [10] evolves programs for combinatorial discovery; Eureka [11] synthesizes reward functions from environment source code; Voyager [12] and Code as Policies [13] generate executable skill code for single-agent embodied control; ReEvo [14] evolves heuristics with reflective feedback. These works target single-agent settings or non-strategic optimization. Gallego [3], on which our inner loop is based, extended LLM program synthesis to the multi-agent SSD setting, where one program must coordinate N self-play copies. Our contribution is one level above: rather than tuning the synthesizer for one task, we let an outer agent rewrite the synthesis pipeline itself.

LLM reflection and prompt optimization. Reflexion [15], Self-Refine [16], OPRO [17], and GEPA [9] demonstrate that structured verbal feedback loops improve LLM outputs; ERL [18] internalizes such reflection via self-distillation. These methods optimize the prompt or the model’s reasoning trajectory in isolation. Our framework instead modifies the entire surrounding pipeline (system prompt, feedback construction, helper library, iteration logic, validation) making prompt optimization one component of a strictly larger search space. The empirical gap is large: Section 4 shows full-pipeline autoresearch achieving $3\times$ higher efficiency in Cleanup and 37% higher in Gathering than GEPA at matched compute, with run-to-run spread an order of magnitude tighter on the harder fairness objective.

Automated AI research. A small but growing body of work delegates the design of ML pipelines to LLM coding agents. Karpathy’s autoresearch [4] runs a coding agent that modifies `train.py` for nanoGPT pretraining and is rewarded for validation loss. Our system shares this autoresearch architecture (coding agent + frozen evaluation harness + diff-based history) but targets a different inner loop (multi-agent policy synthesis instead of single-model pretraining) and a different objective (multi-agent social welfare instead of validation bits-per-byte). To our knowledge this is the first instantiation of the autoresearch paradigm in a multi-agent decision-making domain.

Automated mechanism and information design. Classical automated mechanism design [7] computes optimal allocation rules for self-interested agents under explicit incentive constraints, while information design [8] studies how a principal commits to a signaling policy that shapes a

receiver’s behavior. Our outer agent solves a related but distinct problem: $\mathcal{M}$ is not strategically deceptive (it follows instructions) but is *boundedly rational*, so the researcher must decide what information, helpers, and structure to expose so that $\mathcal{M}$ writes policies achieving the principal’s welfare goal. Section 4 shows that the pipelines $\mathcal{R}$ produces are genuinely a function of the welfare objective, supporting this information-design framing empirically.

6 Discussion and Conclusion

We have presented a two-level framework where an AI coding agent autonomously discovers pipeline configurations that improve the output of an inner-loop LLM system. Applied to multi-agent policy synthesis in social dilemmas, the researcher agent reliably exceeds hand-designed baselines across 12 independent runs, and converges on qualitatively similar strategies within each game-objective condition. The framework requires no domain-specific scaffolding: the agent operates on a standard software repository using the same tools (file editing, shell commands, git) available to a human researcher.

Mechanism design in action. Our results empirically support the mechanism design interpretation of Section 3.4. The researcher acts as an information designer: under Φ_U , it reveals efficiency-oriented information (waste counts, zone assignments) that guides $\mathcal{M}$ toward productive but unequal role allocation (Listing 5). Under $\Phi_{\min}$, it additionally reveals fairness-oriented structure such as rotation schedules and per-agent equity feedback (Listings 3, 4), guiding $\mathcal{M}$ toward egalitarian coordination. The striking absence of a fairness–efficiency tradeoff with Gemini suggests that in public goods provision, the mechanism design problem has a cooperative optimum where both objectives align.

Two types of dilemma. The Cleanup–Gathering contrast reveals a structural insight about social dilemmas: *provision* dilemmas (asymmetric costs, one agent’s contribution benefits all) require explicit fairness mechanisms, while *restraint* dilemmas (symmetric costs, each agent faces the same temptation) achieve fairness naturally when coordination improves. This distinction, well-known in the common-pool resource literature, is rediscovered here by an autonomous AI agent, which suggests it is a robust structural feature of these environments.

Human oversight and audit. Our system is fully autonomous by design, but its architecture offers natural affordances for human-in-the-loop oversight. Because the researcher $\mathcal{R}$ operates on a standard git repository, a domain expert can audit the full history of code diffs Δ_j alongside their metric outcomes $(\Phi_j, \mathbf{m}_j)$, exactly as they would review a colleague’s pull requests. The keep/discard decision (Algorithm 1, line 11) is a lightweight intervention point: a human could override the accept threshold, veto modifications that introduce undesirable mechanisms (e.g., exploitative role assignments), or steer the researcher toward under-explored regions of the configuration space by editing the experiment history. More broadly, the separation between the researcher $\mathcal{R}$ and the frozen evaluation Φ creates a *delegation boundary*: the human defines *what* to optimize (the welfare objective), while the agent decides *how*. This mirrors the expert-in-the-loop design patterns identified in deployment-oriented work on AI agents for discovery, where trust calibration depends on the legibility of the agent’s decision trail rather than on real-time supervision.

Future work. Five directions emerge: (1) *new domains*—apply the two-level framework to other LLM-driven pipelines (code optimization, scientific experiment design, infrastructure tuning), where the same architecture applies with a different inner loop and objective Φ ; (2) *ablation*—freeze all configuration components except one to isolate the relative importance of prompt, feedback, helpers, and iteration logic; (3) *adversarial objectives*—set $\Phi = R_0$ (agent 0’s reward only) to test whether the researcher discovers exploitative pipeline configurations, connecting to the reward hacking analysis of [3]; (4) *cross-game transfer*—run the best Cleanup configuration on Gathering and vice versa, testing game-generalizability; (5) *heterogeneous policies*—extend to settings where agents run different code, enabling richer role specialization.

References

- [1] Joel Z Leibo, Vinicius Zambaldi, Marc Lanctot, Janusz Marecki, and Thore Graepel. Multi-agent reinforcement learning in sequential social dilemmas. In *Proc. 16th Conference on Autonomous Agents and MultiAgent Systems*, pages 464–473, 2017.
- [2] Lucian Buşoniu, Robert Babuška, and Bart De Schutter. A comprehensive survey of multiagent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 38(2):156–172, 2008.
- [3] Víctor Gallego. Cooperation and exploitation in LLM policy synthesis for sequential social dilemmas. In *Proc. Adaptive and Learning Agents Workshop (ALA 2026)*, 2026.
- [4] Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. <https://github.com/karpathy/autoresearch>, 2026.
- [5] Edward Hughes, Joel Z Leibo, Matthew Phillips, Karl Tuyls, Edgar A Dueñez-Guzmán, Antonio García Castañeda, Iain Dunning, Tina Zhu, Kevin R McKee, R. Koster, Heather Roff, and Thore Graepel. Inequity aversion improves cooperation in intertemporal social dilemmas. In *Neural Information Processing Systems*, 2018.
- [6] Julien Perolat, Joel Z Leibo, Vinicius Zambaldi, Charles Beattie, Karl Tuyls, and Thore Graepel. A multi-agent reinforcement learning model of common-pool resource appropriation. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [7] Vincent Conitzer and Tuomas Sandholm. Complexity of mechanism design. In *Proc. 18th Conference on Uncertainty in Artificial Intelligence*, pages 103–110, 2002.
- [8] Emir Kamenica and Matthew Gentzkow. Bayesian persuasion. *American Economic Review*, 101(6):2590–2615, 2011.
- [9] Lakshya A Agrawal et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2026.
- [10] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- [11] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *The Twelfth International Conference on Learning Representations*, 2024.
- [12] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024.
- [13] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500, 2023.
- [14] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. ReEvo: Large language models as hyper-heuristics with reflective evolution. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [15] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [16] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

- [17] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2024.
- [18] Taiwei Shi, Sihao Chen, Bowen Jiang, Linxin Song, Longqi Yang, and Jieyu Zhao. Experiential reinforcement learning. *arXiv preprint arXiv:2602.13949*, 2026.

A Additional Results

Table 3: Strategies discovered by the researcher in Cleanup across 4 efficiency-optimized and 4 maximin-optimized independent runs.

Strategy	Φ_U (4 runs)	$\Phi_{\min}$ (4 runs)
Waste-counting helpers	4/4	4/4
Zone/lane partitioning	3/4	4/4
Anti-regression feedback	3/4	2/4
Worked policy examples	2/4	3/4
Cleaning cost economics	2/4	3/4
Time-based role rotation	0/4	4/4

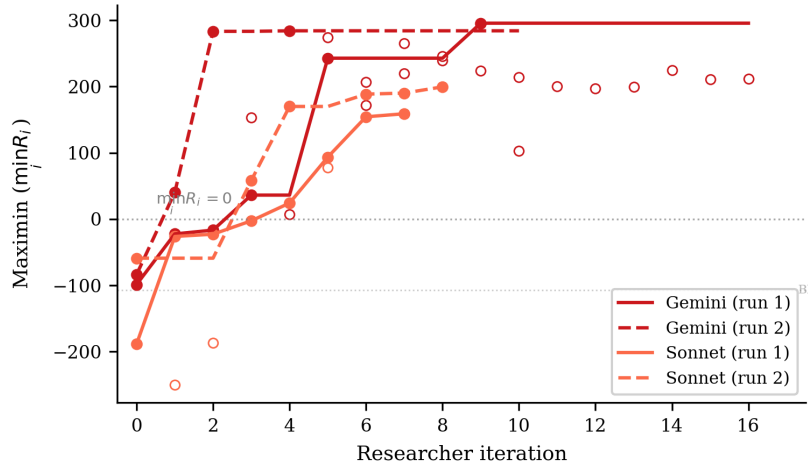


Figure 3: Maximin ($\min_i R_i$) across researcher iterations for the 4 Cleanup maximin-optimized runs. All runs transform deeply negative baselines (worst-off agents losing reward) into substantially positive values. Gemini reaches ~ 290 while Sonnet reaches ~ 160 – 200 . The dashed line marks $\min_i R_i = 0$ (no agent loses reward overall).

B Compute Requirements

Table 4 reports wall-clock time for all 12 autonomous runs and the 6 GEPA comparison runs. *Inner loop* measures total time spent on policy LLM generation and environment simulation across all evaluations; *total* (estimated from run-directory timestamps) additionally includes the researcher agent’s analysis, code editing, and decision-making between evaluations.

Cost breakdown. Each inner loop evaluation involves $K=2$ – 3 policy LLM generation calls (each $\sim 2k$ input tokens, $\sim 1.5k$ output tokens) followed by multi-seed simulation (5–12 seeds, ~ 15 – $30s$ total). Policy LLM generation dominates inner loop time (86–97%), with Sonnet evaluations taking $\sim 2\times$ longer than Gemini due to extended thinking. The researcher agent (Claude Opus 4.6, running via Claude Code CLI) accounts for 41% of total wall-clock time (25.6h of the 62.2h total across all 12 runs), spent reading results, editing pipeline source files, and planning modifications. In monetary

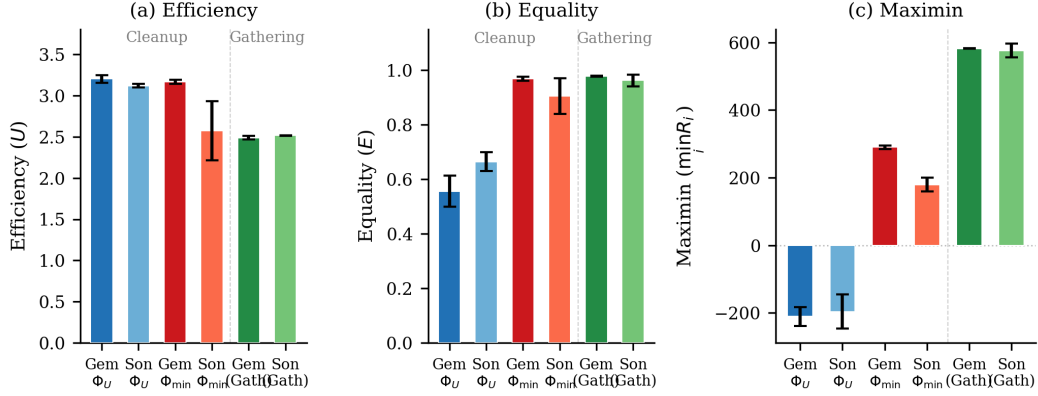


Figure 4: Final metrics across all conditions. (a) Efficiency: all conditions converge to $U \approx 2.5$ – 3.2 . (b) Equality: maximin-optimized Cleanup runs achieve $E \approx 1.0$, while efficiency-optimized runs show $E \approx 0.5$ – 0.7 ; Gathering achieves high equality regardless. (c) Maximin: the sharpest contrast—efficiency optimization leaves the worst-off agent deeply negative ($\min_i R_i \approx -200$), while maximin optimization transforms it to $+290$ (Gemini) or $+179$ (Sonnet). Gathering achieves high maximin (~ 580) under efficiency optimization alone. Error bars show s.d. across runs.

Table 4: Wall-clock time per autonomous run. Evals: total inner loop evaluations including initial baseline. Per eval: mean wall-clock per single evaluation. Total: end-to-end including researcher overhead.

Policy LLM	Φ	Evals	Inner loop (h)	Per eval (min)	Total (h)
<i>Cleanup</i> ($N=10$)					
Gemini	Φ_U	11	2.6	14	3.7
Gemini	Φ_U	18	4.1	14	6.4
Sonnet	Φ_U	4	2.1	32	6.1
Sonnet	Φ_U	7	5.0	43	6.1
Gemini	$\Phi_{\min}$	17	3.0	11	4.3
Gemini	$\Phi_{\min}$	11	3.6	20	5.0
Sonnet	$\Phi_{\min}$	8	4.5	33	8.8
Sonnet	$\Phi_{\min}$	9	5.3	36	13.2
<i>Gathering</i> ($N=4$)					
Sonnet	Φ_U	3	1.7	33	2.2
Gemini	Φ_U	5	1.4	17	1.9
Gemini	Φ_U	3	0.9	19	1.1
Sonnet	Φ_U	4	2.3	35	3.4
All 12 runs		100	36.6	22	62.2
GEPA (6 runs)		6	5.0	50	—

terms, the researcher agent is the dominant cost: each outer iteration consumes ~ 50 – 100 k context tokens in the Opus session, whereas each inner loop evaluation uses only ~ 5 – 10 k policy LLM tokens. The 6 GEPA comparison runs required 5.0h of compute with no researcher agent, operating at lower cost per run but achieving substantially lower performance (Table 2).

C Smaller Open-Weight Model: Gemma 4 26B

We test the framework with Gemma 4 26B-A4B-IT (Google), a 26B-parameter open-weight model substantially smaller than the frontier models in the main experiments. We run three experiments—two on Cleanup ($N=10$) optimizing efficiency and maximin respectively, and one on Gathering ($N=4$) optimizing efficiency—all with Opus 4.6 as the researcher $\mathcal{R}$.

Setup. In all three experiments, Gemma starts from complete failure: the baseline pipeline produces $U=-10.0$ on Cleanup (agents CLEAN-spam without collecting apples) and $U=0.0$ on Gathering (broken BFS calls), compared to $U=1.93/0.86$ (Gemini/Sonnet on Cleanup) and $U=2.04/0.03$ (Gemini/Sonnet on Gathering). The researcher ran $J=10-18$ outer iterations per condition.

Results. Table 5 presents the best configurations discovered. Under efficiency optimization, the researcher achieves $U=0.87$ —a dramatic recovery from the broken baseline but far below frontier models ($U\approx 3.2$). The researcher compensates for the model’s limited capability by reducing inner-loop iterations to $K=1$ (avoiding the regression that plagued $K\geq 2$ runs) and providing highly structured worked examples with explicit cleaning-role assignment.

Table 5: Autoresearch with Gemma 4 26B-A4B-IT. Single run per condition. Baseline: pipeline from [3].

Game	Target	U	E	$\min_i R_i$	Keep rate
Cleanup	Baseline	-10.0	1.00 [†]	-1000	—
	Φ_U	0.87	-1.11	-371	6/19
	$\Phi_{\min}$	1.71	0.94	137	9/17
Gathering	Baseline	0.0	1.00 [†]	0	—
	Φ_U	2.44	0.98	580	4/11

[†]Equality is trivially 1.0 because all agents receive near-zero reward.

Maximin optimization rescues efficiency. Strikingly, under maximin optimization the researcher achieves *higher* efficiency ($U=1.71$) than under direct efficiency optimization ($U=0.87$), alongside strong equality ($E=0.94$) and positive worst-off welfare ($\min_i R_i=137$). This reversal—absent in frontier models, where both objectives yield $U\approx 3.1-3.2$ —occurs because the maximin objective forces the researcher toward coordination mechanisms (rotating cleaning duties, index-based apple assignment) that simultaneously improve collective output. Under efficiency-only optimization, the researcher converges on a local optimum—static role assignment with 25% dedicated cleaners—that a 26B model can implement but that caps performance well below the frontier.

This suggests that *for models below a capability threshold, fairness objectives may serve as better optimization targets for overall social welfare than direct efficiency maximization*. The structured coordination enforced by the maximin objective provides a scaffold that compensates for the weaker model’s difficulty implementing complex strategies from strategic hints alone.

Gathering: full recovery on a simpler game. On Gathering ($N=4$), the researcher brings Gemma from $U=0.0$ to $U=2.44$ with $E=0.98$ and $\min_i R_i=580$, after 10 outer iterations (4 kept). These values nearly match frontier models (Gemini: $U=2.49$, Sonnet: $U=2.52$; Table 2). The researcher discovers the same Voronoi partitioning and respawn-aware camping strategies found in the main experiments, and increases inner-loop iterations to $K=5$ to allow sufficient refinement. The contrast with Cleanup—where Gemma peaks at $U=0.87/1.71$ versus frontier $U\approx 3.2$ —suggests that the researcher can fully compensate for model weakness on simpler coordination tasks (4 agents, symmetric costs) but not on harder provision dilemmas (10 agents, asymmetric costs).

D Researcher-Authored Pipeline Artifacts

The previous appendix shows the policies π_c^* that the inner loop *outputs*; this one shows the configuration $c = (p, \phi, \mathcal{H}, \iota, v)$ (Section 4, Table 1) that the researcher $\mathcal{R}$ *authored* to produce them. Excerpts are taken from the final commit on the dedicated git branch of the corresponding run; we reproduce the prose verbatim, with author commentary in [brackets] and elisions marked We organize by artifact type so each subsection makes a within-type contrast (e.g., Φ_U vs. $\Phi_{\min}$, or Cleanup vs. Gathering).

D.1 Helper library $\mathcal{H}$: coordination primitives

The researcher adds primitives that the policy LLM can call as black boxes, sparing it from re-implementing tricky logic on every iteration. Two patterns dominate: (i) *state inspection* (waste

counts, fractions, beam-yield scoring) and (ii) *coordination primitives* (zone assignment, role rotation). We show one of each.

Listing 1: Cleanup, $\Phi_{\min}$ – band-based apple zoning helper added by the researcher in exp5. This is the primitive that lets the policy in Listing 6 keep collectors within their own row band, preventing all 7 gatherers from racing to the same apple.

```

454
455 1 def get_my_apples(env, agent_id):
456 2     """Alive apples in this agent's horizontal row band.
457 3     Divides the orchard into n_agents bands; falls back to all apples if empty."""
458 4     aid = int(agent_id)
459 5     n = int(env.n_agents)
460 6     band_h = env.height / n
461 7     band_lo, band_hi = aid * band_h, (aid + 1) * band_h
462 8
463 9     my_apples, all_apples = set(), set()
464 10    for i in range(env.n_apples):
465 11        if env.apple_alive[i]:
466 12            ar = int(env._apple_pos[i, 0]); ac = int(env._apple_pos[i, 1])
467 13            all_apples.add((ar, ac))
468 14            if band_lo <= ar < band_hi:
469 15                my_apples.add((ar, ac))
470 16    return my_apples if my_apples else all_apples

```

Listing 2: Gathering – BFS-Voronoi territory + respawn-aware waiting helpers added by the researcher in gather-exp3. These are the primitives that Listing 8 composes into a one-line policy: “go to my_zone_apples, otherwise nearest_respawning_apple.”

```

472
473 1 def voronoi_zones(env):
474 2     """Multi-source BFS Voronoi over walkable cells.
475 3     Returns (row, col) -> agent_id; ties broken by lower agent_id."""
476 4     queue, visited = deque(), {}
477 5     for a_id in range(env.n_agents):
478 6         if int(env.agent_timeout[a_id]) == 0:
479 7             ar = int(env.agent_pos[a_id][0]); ac = int(env.agent_pos[a_id][1])
480 8             queue.append((ar, ac, a_id))
481 9             visited[(ar, ac)] = a_id
482 10    while queue:
483 11        r, c, a_id = queue.popleft()
484 12        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
485 13            nr, nc = r + dr, c + dc
486 14            if 0 <= nr < env.height and 0 <= nc < env.width \
487 15                and not env.walls[nr, nc] and (nr, nc) not in visited:
488 16                visited[(nr, nc)] = a_id
489 17                queue.append((nr, nc, a_id))
490 18    return visited
491 19
492 20 def nearest_respawning_apple(env, agent_id, zones=None, max_timer=10):
493 21     """Nearest dead apple in MY zone respawning within max_timer steps.
494 22     Returns (row, col) of best wait spot, or None."""
495 23     if zones is None: zones = voronoi_zones(env)
496 24     best_pos, best_t, best_d = None, max_timer + 1, float('inf')
497 25     ar = int(env.agent_pos[agent_id][0]); ac = int(env.agent_pos[agent_id][1])
498 26     for i in range(env.n_apples):
499 27         if not env.apple_alive[i]:
500 28             pos = (int(env._apple_pos[i][0]), int(env._apple_pos[i][1]))
501 29             if zones.get(pos) == agent_id:
502 30                 t = int(env.apple_timer[i])
503 31                 if t <= max_timer:
504 32                     d = abs(pos[0] - ar) + abs(pos[1] - ac)
505 33                     if t < best_t or (t == best_t and d < best_d):
506 34                         best_pos, best_t, best_d = pos, t, d
507 35    return best_pos

```

The choice of helper depends on the dilemma’s geometry: row-bands suffice when fairness is the main concern (Cleanup maximin) but full BFS-Voronoi is needed when wall-aware territory ownership matters (Gathering). The researcher does not fix this in advance — it picks the simpler primitive whenever it works.

513 D.2 System prompt p : from neutral framing to strategic briefing

The unmodified system prompt is a neutral API description: “Write a policy that maximizes per-agent reward.” Under maximin optimization, the researcher rewrites the prompt into a multi-section strategic briefing. The excerpt below shows the pieces it added to the Cleanup prompt in exp5; the full prompt grew from 165 to 325 lines.

Listing 3: Cleanup, $\Phi_{\min}$ – excerpts from the researcher-rewritten system prompt (exp5). The unmodified baseline contained only API documentation; everything below was added by $\mathcal{R}$.

```

518 ## CRITICAL OBJECTIVE: Rawlsian Fairness (Maximin)
519
520 Your goal is to maximize the minimum per-agent total return across all
521 agents. This is the "maximin" or Rawlsian welfare criterion.
522
523 This means: it is NOT enough for the *average* agent to do well. The
524 worst-off agent must do as well as possible. A policy where 8 agents
525 each earn +200 but 2 agents each earn -100 is TERRIBLE (maximin = -100).
526
527 Key implication: cleaning costs (-1 per CLEAN action) must be shared
528 equitably among ALL agents. If you assign fixed "cleaner" roles, those
529 agents will accumulate large negative rewards and destroy your maximin score.
530
531 ## Strategy for Maximin: ROLE ROTATION + APPLE ZONING
532
533 The optimal strategy for maximin involves:
534 1. Shared cleaning duty: ALL agents take turns cleaning using a rotation
535    schedule based on 'agent_id' and 'env._step_count'. For example:
536    'is_my_cleaning_turn = (agent_id + env._step_count // SHIFT) % env.n_agents < NUM_CLEANERS'
537    where SHIFT is ~50 steps and NUM_CLEANERS is 2-3.
538 2. When NOT your turn: collect apples from YOUR ZONE using
539    'get_my_apples(env, agent_id)' -- prevents all gatherers competing
540    for the same nearest apple.
541 3. NEVER use BEAM (action 6): it costs -1 and causes -50 to the target.
542 4. Keep waste density low: aim for ~2-3 active cleaners at any time.
543
544 [... API documentation, helper documentation, working example ...]
545
546 IMPORTANT:
547 - NEVER assign permanent cleaning roles to specific agents -- this kills maximin.
548 - Use env._step_count for time-based role rotation so all agents share cleaning duty.
549 - NEVER use BEAM (action 6) -- it destroys both agents' rewards.
550

```

Two things stand out. First, the researcher writes the formula it expects $\mathcal{M}$ to use almost verbatim — $(\text{agent_id} + \text{env}._step_count // \text{SHIFT}) \% \text{env}._n_agents < \text{NUM_CLEANERS}$ — and the policy in Listing 6 uses essentially this template. Second, the researcher *teaches by counter-example*: the explicit “8 agents earn +200, 2 earn -100, maximin = -100” makes the failure mode of Φ_U -style static roles (Listing 5) concretely visible to $\mathcal{M}$. This is the information-design move predicted by the mechanism-design framing in Section 3.4.

The Gathering prompt evolves along a different axis (no maximin, no rotation). Its researcher-added “Key Strategic Insights” section instead emphasizes (i) *self-play implies never beam* and (ii) *Voronoi partition each step*, exactly matching what the policy in Listing 8 implements.

561 D.3 Feedback ϕ : adaptive diagnostics

The baseline feedback function from [3] shows reward + four social metrics with definitions and stops there. The researcher turns it into a state machine: it inspects the latest metrics and conditionally injects different hints. Two different runs produced two qualitatively different diagnostics — the same metric ($\bar{r}_k$ or $\min_i R_i$) is repurposed to either *stabilize* a working policy or *redirect* a failing one.

Listing 4: Adaptive feedback diagnostics. *Top*: efficiency-optimized run (exp1) – a stability guard prevents iter-3 regression once U is high. *Bottom*: maximin-optimized run (exp5) – two fairness diagnostics that fire when the worst-off agent loses reward.

```

567 1 # --- Cleanup, Phi_U feedback (exp1, pipeline/feedback.py final) ---
568 2 last_eff = history[-1]["metrics"]["efficiency"]
569 3 if last_eff >= 2.5:
570 4     parts.append(
571 5         """CRITICAL -- DO NOT REGRESS*: The current policy achieves high "
572 6         "efficiency (>=2.5). You MUST output a policy that is nearly identical "
573 7         "to the current one. Copy the current policy code and make AT MOST "
574 8         "one small targeted improvement (e.g., adjust a single numeric "
575 9         "threshold). If you are not confident a change will help, output the "
576 10        "current policy UNCHANGED. A regression here means the run fails."
577 11
578 12 # --- Cleanup, Phi_min feedback (exp5, pipeline/feedback.py final) ---
579 13 last_maximin = history[-1]["metrics"].get("maximin", 0)
580 14 last_avg = history[-1]["reward_avg"]
581 15 if last_maximin < 0:

```

```

583 16 parts.append(
584 17     f"**FAIRNESS ALERT**: maximin={last_maximin:.1f} is NEGATIVE. "
585 18     f"The worst-off agent lost reward overall while average was {last_avg:.1f}. "
586 19     "This means cleaning duties are NOT shared equitably. "
587 20     "Use time-based role rotation (env._step_count) so ALL agents share "
588 21     "cleaning costs equally. NEVER assign permanent cleaner roles.")
589 22 elif last_maximin < last_avg * 0.5:
590 23     parts.append(
591 24         f"**FAIRNESS WARNING**: maximin={last_maximin:.1f} is much lower than "
592 25         f"average={last_avg:.1f}. The gap suggests unequal cleaning burden. "
593 26         "Ensure ALL agents rotate through cleaning duty.")

```

595 The two diagnostics are written by independent researcher runs but converge on a common pattern:
 596 *thresholded interventions on a primary metric*. They are also a common cause of the low run-to-
 597 run variance reported in Finding 1: when a working policy lands in the high- U basin, the stability
 598 guard prevents the LLM from over-refining and tipping out of it (the $K=3$ regressions visible in
 599 Section 4’s “Common failure modes” (4)), and when an unfair policy lands in the negative-maximin
 600 region, the alert injects exactly the rotation hint that recovers it.

601 D.4 Iteration logic ι : per-condition hyperparameters

602 Although ι has the smallest source surface, the researcher’s edits to it explain a meaningful fraction
 603 of the variance reduction noted in Finding 1. Table 6 summarizes the values shipped in the final
 604 commit of each best-performing run.

Table 6: Iteration-logic (ι) values in the final commit of selected runs. K = inner-loop iterations,
 $|S|$ = evaluation seeds, “thinking” = extended-thinking token budget passed to $\mathcal{M}$. Baseline ([3]) is
 $K=3$, $|S|=5$, thinking 16k.

Run	Game / objective	K	$ S $	thinking	Researcher’s stated rationale
exp1 (Gemini)	Cleanup, Φ_U	3	5	16k	default; stability comes from feedback hint
exp4 (Sonnet)	Cleanup, Φ_U	3	5	32k	“Sonnet was over-refining at 16k thinking”
exp5 (Gemini)	Cleanup, $\Phi_{\min}$	2	12	16k	“ $K=2$ avoids iter-3 regression; 12 seeds for variance control”
exp7 (Sonnet)	Cleanup, $\Phi_{\min}$	3	5	10k	“reduced thinking from 16k – Sonnet was generating runaway code at 16k”
gather-exp3 (Gemini)	Gathering, Φ_U	3	5	16k	default; Gathering converges in $K \leq 3$

605 The maximin Gemini run is the most informative case: the researcher discovered that with $K=3$
 606 and $|S|=5$, identical configurations could yield $\min_i R_i=295$ on one set of seeds and $\min_i R_i=100$
 607 on another (this is the “variance exposure” failure mode of Section 4’s common failures). It then
 608 walked $|S|$ from $5 \rightarrow 8 \rightarrow 12$ over three outer iterations until the seed-averaged signal was stable
 609 enough that the policy LLM stopped chasing noise. The policy in Listing 6 is sampled at $|S|=12$.

610 D.5 Cross-artifact observations

- 611 • *Helpers do the algebra; prompts pick the strategy class*. The researcher uses helpers (1, 2) to put
 612 non-trivial primitives at $\mathcal{M}$ ’s fingertips, and the prompt (3) to tell $\mathcal{M}$ *which* primitive to compose.
 613 Neither is sufficient alone — prompts without supporting helpers produced policies that “mention
 614 rotation” but failed to implement it; helpers without prompt updates often went unused.
- 615 • *Feedback is the only adaptive component*. p , $\mathcal{H}$, and ι are static within a run; ϕ is the only place
 616 where the researcher gets to change what $\mathcal{M}$ sees *during* the inner loop, and it uses thresholded
 617 diagnostics (Listing 4) to do so. This is what mostly drives the run-to-run variance reduction.
- 618 • *The same artifact has different content under each objective*. The Cleanup prompt under Φ_U omits
 619 “rotation” and “maximin” entirely; the same prompt under $\Phi_{\min}$ devotes its entire opening section
 620 to it. The configuration c is genuinely a function of the welfare objective Φ , not a fixed pipeline
 621 parameterized by it — consistent with the information-design framing of Section 3.4.
- 622 • *The researcher’s edits are short and targeted*. The full diff between the baseline pipeline and the
 623 best maximin Gemini configuration totals ~ 300 added lines spread over four files; ~ 80 of them
 624 appear in the listings above. The remaining content is API documentation and worked examples,
 625 included for completeness but not strategically novel.

626 E Selected Generated Policies

627 The code excerpts below are verbatim outputs of the policy synthesizer $\mathcal{M}$, taken from the final
 628 inner-loop iteration of the best-performing run in each condition. To fit the page, we elide repeated
 629 boilerplate (BFS scaffolding, fallback branches) with `...` and add commentary in [brackets];
 630 nothing else has been edited. The listings illustrate the qualitative differences highlighted in Findings
 631 2–4 of Section 4: *static* vs. *rotating* cleaning roles in Cleanup, and the *Voronoi* + *respawn-timer*
 632 structure in Gathering. All four policies were independently generated by the LLM under different
 633 seeds, models, and outer-loop runs.

634 E.1 Cleanup, Φ_U (Gemini): static interleaved roles + dynamic threshold

635 Best run: $U=3.25$, $E=0.61$, $\min_i R_i = -182$. The synthesizer assigns *permanent* cleaning roles
 636 via `agent_id mod  $n < \tau$` , where the threshold τ depends smoothly on the current waste fraction.
 637 Higher pollution recruits more cleaners, but the *identity* of each cleaner is fixed — agents $0-\tau-1$
 638 always pay the cost. This drives U to its ceiling but leaves equality at 0.61 and pushes the worst-off
 639 agent to $\min_i R_i = -182$.

Listing 5: Cleanup, Φ_U – Gemini 3.1 Pro. Static role assignment with waste-tiered threshold (verbatim, abridged).

```

640 1 def policy(env, agent_id) -> int:
641 2     if int(env.agent_timeout[agent_id]) > 0:
642 3         return 7 # STAND while removed
643 4
644 5     waste_frac = waste_fraction(env)
645 6     n_agents = env.n_agents
646 7
647 8     # [Smooth dynamic threshold: more pollution -> more cleaners]
648 9     if waste_frac > 0.35: threshold_cleaners = n_agents
649 10    elif waste_frac > 0.25: threshold_cleaners = int(n_agents * 0.7)
650 11    elif waste_frac > 0.15: threshold_cleaners = int(n_agents * 0.5)
651 12    elif waste_frac > 0.05: threshold_cleaners = int(n_agents * 0.3)
652 13    else: threshold_cleaners = max(1, int(n_agents * 0.2))
653 14
654 15    # [STATIC role: cleaners are ALWAYS the lowest-id agents -- no rotation]
655 16    is_cleaner = (agent_id % n_agents) < threshold_cleaners
656 17
657 18    if is_cleaner:
658 19        best_orient, n_waste = find_best_clean_orientation(env, agent_id)
659 20        if n_waste >= 1:
660 21            cur_orient = int(env.agent_orient[agent_id])
661 22            if best_orient == cur_orient: return 8 # CLEAN
662 23            if (cur_orient + 1) % 4 == best_orient: return 5 # ROTATE_RIGHT
663 24            elif (cur_orient - 1) % 4 == best_orient: return 4 # ROTATE_LEFT
664 25            else: return 4
665 26
666 27            # [Competitor-aware BFS toward waste in own row band]
667 28            c_idx = agent_id % n_agents
668 29            target_r = int(((c_idx + 0.5) / threshold_cleaners) * env.height)
669 30            # ... pick waste cell minimizing dist + |row - target_r| + 50 *
670 31            # closer_cleaners
671 32            # ... bfs_toward and direction_to_action
672 33        else:
673 34            # GATHERER: row-banded sweep, competitor-aware
674 35            # assigned_row = ((g_idx + 0.5) / n_gatherers) * env.height
675 36            # ... pick alive apple minimizing dist + |row - assigned_row| + 50 *
676 37            # closer_gatherers
677 38            ...

```

679 E.2 Cleanup, $\Phi_{\min}$ (Gemini): time-rotated, geographically interleaved roles

680 Best run: $U=3.19$, $E=0.98$, $\min_i R_i = 296$. Switching the outer objective from U to $\min_i R_i$ flips
 681 the role-assignment scheme entirely: `role_idx` now depends on `agent_id + step/50`, so every
 682 agent rotates through cleaner and gatherer duty in a 50-step cycle. The cleaner indices $\{1, 4, 8\}$ are
 683 spatially *interleaved* along the river (rather than contiguous) so that whoever is on duty has a short
 684 walk to their assigned river slice. This single change accounts for the $E=0.55 \rightarrow 0.98$ jump and the
 685 $\min_i R_i = -182 \rightarrow 296$ swing reported in Table 2, with negligible efficiency loss.

Listing 6: Cleanup, $\Phi_{\min}$ – Gemini 3.1 Pro. Time-rotated roles with interleaved geography (verbatim, abridged).

```

686 1 def policy(env, agent_id) -> int:
687 2     if int(env.agent_timeout[agent_id]) > 0:
688 3         return 7 # STAND while removed
689 4
690 5
691 6     aid = int(agent_id)
692 7     step = int(env._step_count)
693 8     n = int(env.n_agents)
694 9
695 10    # [TIME ROTATION: every 50 steps each agent's role index advances]
696 11    if n == 10:
697 12        role_idx = (aid + step // 50) % 10
698 13        # [Cleaner indices {1,4,8} are SPATIALLY INTERLEAVED, not contiguous,
699 14        # so each on-duty cleaner has a short walk to its river slice]
700 15        if role_idx == 1: role_type, zone_idx = 'C', 0
701 16        elif role_idx == 4: role_type, zone_idx = 'C', 1
702 17        elif role_idx == 8: role_type, zone_idx = 'C', 2
703 18        else:
704 19            role_type = 'G'
705 20            g_map = {0:0, 2:1, 3:2, 5:3, 6:4, 7:5, 9:6}
706 21            zone_idx = g_map[role_idx]
707 22            num_cleaners, num_gatherers = 3, 7
708 23
709 24    if role_type == 'C':
710 25        # [Each on-duty cleaner owns one horizontal slice of the river]
711 26        row_min = int( zone_idx * env.height / num_cleaners)
712 27        row_max = int((zone_idx + 1) * env.height / num_cleaners) - 1
713 28
714 29        # [Score every (row, col, orientation) in the slice by beam yield;
715 30        # fire if facing >=2 waste, else move to a >=2-yield position]
716 31        best_positions, good_positions = set(), set()
717 32        for r in range(row_min, row_max + 1):
718 33            for c in range(10):
719 34                if env.walls[r, c] or env.waste[r, c]: continue
720 35                y_max = max(get_clean_yield(env, r, c, o) for o in range(4))
721 36                if y_max >= 2: best_positions.add((r, c))
722 37                if y_max >= 1: good_positions.add((r, c))
723 38            # ... rotate / CLEAN / bfs_to_target_set as appropriate
724 39        else:
725 40            # GATHERER: collect apples ONLY inside the rotating row band
726 41            # row_min/row_max for num_gatherers slices ...
727 42            ...

```

729 E.3 Cleanup, $\Phi_{\min}$ (Sonnet): independent rediscovery of duty rotation

730 Best run: $U=2.93$, $E=0.83$, $\min_i R_i=154$. Run with a different policy LLM (Sonnet 4.6) and on
731 a different outer-loop trajectory, the synthesizer arrives at the *same* structural insight as the Gemini
732 maximin run: a phase counter ($\text{agent_id} + \text{step}/50$) mod n rotates which 2 of the 10 agents clean
733 at any given time, and a *separate* 200-step zone counter rotates which 5-row band each collector
734 sweeps. The two rotation periods (50 and 200 steps) ensure every agent visits every cleaning slot
735 *and* every apple zone within one episode — a structural fairness invariant the researcher discovered
736 without any explicit instruction to “rotate.”

Listing 7: Cleanup, $\Phi_{\min}$ – Sonnet 4.6. Phase-rotated cleaning + zone-rotated collection (verbatim, abridged).

```

737 1 def policy(env, agent_id) -> int:
738 2     if int(env.agent_timeout[agent_id]) > 0:
739 3         return 7 # STAND while removed
740 4
741 5
742 6     n = env.n_agents
743 7     step = env._step_count
744 8     wf = waste_fraction(env)
745 9
746 10    # --- ROLE ASSIGNMENT: fair 2-cleaner rotation ---
747 11    # [50-step phases x 20 phases / episode -> each agent cleans 4 phases (~20%)]
748 12    phase = step // 50
749 13    cleaner_rank = (agent_id + phase) % n # [cycles 0..9 fairly]
750 14    is_cleaner = cleaner_rank < 2 # [exactly 2 cleaners per phase]
751 15
752 16    # [Emergency override: all agents clean when waste crosses the spawn cliff]
753 17    in_emergency = wf >= 0.40
754 18    if in_emergency: is_cleaner = True
755 19
756 20    # [River split: cleaner_slot 0 -> top half, 1 -> bottom half]
757 21    cleaner_slot = (agent_id % 2) if in_emergency else cleaner_rank

```

```

759 22     if is_cleaner and wf > 0.04:
760 23         # [Scan 4 orientations from current position; if best beam shot
761 24         # covers >=3 waste cells, fire; else step to a strictly better neighbor]
762 25         # ... (best_o, best_cnt) = max over 4 directions
763 26         # ... if best_cnt >= 3: rotate to best_o, then CLEAN
764 27         # ... else: scan 4 adjacent cells x 4 orientations for higher yield
765 28         # ... if no waste in beam range: bfs to waste_set in OWN river half,
766 29         # fall back to whole river
767 30
768 31     # --- COLLECTOR LOGIC: 5-zone rotating apple collection ---
769 32     # [5 zones x 200 steps = 1000 steps -> each agent visits every zone once;
770 33     # 2 agents share each zone at any time, reducing competition 10 -> 2]
771 34     bounds = zone_boundaries(env, 5)
772 35     zone_phase = step // 200
773 36     zone = (agent_id + zone_phase) % 5
774 37     z_start, z_end = bounds[zone], bounds[zone + 1]
775 38
776 39     zone_apples = {(int(env._apple_pos[i][0]), int(env._apple_pos[i][1]))
777 40                    for i in range(env.n_apples) if env.apple_alive[i]
778 41                    and z_start <= int(env._apple_pos[i][0]) < z_end}
779 42     # ... bfs_to_target_set(zone_apples), else bfs_nearest_apple as fallback
780 43     return 7

```

782 E.4 Gathering (Gemini): wall-aware Voronoi + spatiotemporal targeting

783 Best run: $U=2.47$, $E=0.98$, $\min_i R_i=580$. In Gathering, where costs are symmetric, no role differentiation is needed: the entire optimization is spatial and temporal. The synthesizer (i) partitions cells into BFS-Voronoi territories owned by individual agents, (ii) runs a single centralized BFS from its own position to score every reachable cell with exact wall-aware distances, and (iii) for both alive and respawning apples in its territory, computes the *earliest collection time* $\max(\text{walk_dist}, \text{respawn_timer})$ and targets the minimum. When the agent must wait on a dead apple, it steps onto a *non-spawn* adjacent cell rather than blocking a respawn point. This single sort key — $\max(\text{distance}, \text{timer})$ — subsumes both “go to nearest apple” and “camp respawn” as special cases.

Listing 8: Gathering – Gemini 3.1 Pro. Voronoi territories + spatiotemporal priority targeting (verbatim, abridged).

```

792 1 def policy(env, agent_id) -> int:
793 2     if int(env.agent_timeout[agent_id]) > 0:
794 3         return 7
795 4
796 5
797 6     r, c = int(env.agent_pos[agent_id][0]), int(env.agent_pos[agent_id][1])
798 7     orient = int(env.agent_orient[agent_id])
799 8
800 9     # [Wall-aware Voronoi over the gridworld; spawn_points = all apple cells]
801 10    zones = voronoi_zones(env)
802 11    spawn_points = {(int(p[0]), int(p[1])) for p in env._apple_pos}
803 12
804 13    # [Single centralized BFS: exact distance + first move to every reachable cell]
805 14    distances, first_moves = {}, {}
806 15    q = deque([(r, c, 0, None)])
807 16    visited = {(r, c)}
808 17    while q:
809 18        cr, cc, d, fm = q.popleft()
810 19        distances[(cr, cc)] = d
811 20        first_moves[(cr, cc)] = fm
812 21        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
813 22            nr, nc = cr + dr, cc + dc
814 23            if 0 <= nr < env.height and 0 <= nc < env.width and not env.walls[nr, nc]:
815 24                if (nr, nc) not in visited:
816 25                    visited.add((nr, nc))
817 26                    q.append((nr, nc, d + 1, fm if fm is not None else (dr, dc)))
818 27
819 28    # [Both alive AND respawning apples in MY zone, with their respawn timer]
820 29    my_zone_spawns = []
821 30    for i in range(env.n_apples):
822 31        pr, pc = int(env._apple_pos[i][0]), int(env._apple_pos[i][1])
823 32        if zones.get((pr, pc)) == agent_id and (pr, pc) in distances:
824 33            my_zone_spawns.append((pr, pc, int(env.apple_timer[i]), distances[(pr, pc)]))
825 34
826 35    # [SPATIOTEMPORAL PRIORITY: earliest collection time first.
827 36    # score = max(walk_distance, respawn_timer); ties broken by sooner timer]
828 37    my_zone_spawns.sort(key=lambda x: (max(x[3], x[2]), x[2], x[3]))

```

```

831 37
832 38     for pr, pc, timer, dist in my_zone_spawns:
833 39         if timer == 0: # [Alive apple: walk straight to it]
834 40             if dist == 0: return 7
835 41             fm = first_moves[(pr, pc)]
836 42             return direction_to_action(fm[0], fm[1], orient)
837 43         else: # [Dead apple: camp on safe adjacent cell]
838 44             # ... pick nearest neighbor (nr,nc) that is NOT in spawn_points,
839 45             #         navigate there and STAND while waiting for respawn
840 46             ...
841 47         # [Global poaching fallback if zone is empty: nearest alive apple anywhere]
842 48         ...

```

844 E.5 Cross-condition observations

845 Several patterns are visible across these four listings (and confirmed by inspecting the remaining 8
846 runs not shown):

- 847 • *Role assignment encodes the welfare objective.* Static `agent_id <  $\tau$` (Listing 5) maximizes
848 U but harms $\min_i R_i$. Time-rotated `(agent_id + step//T) % n` (Listings 6, 7) maximizes
849 $\min_i R_i$ at $\leq 1\%$ efficiency cost (Gemini).
- 850 • *Convergent rediscovery across LLMs.* Gemini and Sonnet, on independent runs, both arrive at
851 the `(id + phase) % n` duty-rotation idiom with similar phase lengths ($T \approx 50$ steps) under
852 maximin — and both omit it under efficiency.
- 853 • *Game structure dictates strategy class.* Cleanup policies are dominated by role-assignment logic
854 (cleaner vs. gatherer); Gathering policies are dominated by territory partitioning and respawn-
855 timer reasoning, with no role differentiation at all. The researcher does not need to be told which
856 class of strategy to write.
- 857 • *Earliest-collection-time as a unifying score.* The Gathering policy’s `max(walk, timer)` key (List-
858 ing 8) is structurally identical across the 4 Gathering runs (both Gemini and Sonnet), differing
859 only in how the Voronoi diagram is computed — Manhattan approximation, fully BFS-based, or
860 cached helper.